

Batch 1A

Traffic Light Sequence - Traffic Light Multi-output sequencing

PROBLEM:

A pre-packaged Traffic Light module connected to pins D9, D10, and D11 cycles through standard phases: Red (Stop) for 4 seconds -> Green (Go) for 4 seconds -> Yellow (Prepare to Stop) for 1.5 seconds -> repeating.

CONNECTION:

GPIO 25 —► RED

GPIO 26 —► YELLOW

GPIO 27 —► GREEN

CODE:

```
const int RED = 25;
```

```
const int YELLOW = 26;
```

```
const int GREEN = 27;
```

```
void setup() {
```

```
    pinMode(RED, OUTPUT);
```

```
    pinMode(YELLOW, OUTPUT);
```

```
    pinMode(GREEN, OUTPUT);
```

```
}
```

```
void loop() {  
  
    // RED - Stop (4 seconds)  
  
    digitalWrite(RED, HIGH);  
  
    digitalWrite(YELLOW, LOW);  
  
    digitalWrite(GREEN, LOW);  
  
    delay(4000);  
  
  
    // GREEN - Go (4 seconds)  
  
    digitalWrite(RED, LOW);  
  
    digitalWrite(YELLOW, LOW);  
  
    digitalWrite(GREEN, HIGH);  
  
    delay(4000);  
  
  
    // YELLOW - Prepare to stop (1.5 seconds)  
  
    digitalWrite(RED, LOW);  
  
    digitalWrite(YELLOW, HIGH);  
  
    digitalWrite(GREEN, LOW);  
  
    delay(1500);  
  
}
```

OBSERVED OUTPUT:

Red LED - Stop

Green LED - Go

Yellow LED - Slow

RED LED – Stop

. . .

. . .

. . .

nth term (Red LED - Stop

Green LED - Go

Yellow LED – Slow)

EXPECTED OUTPUT:

Red LED – Stop — 4 seconds

Green LED – Go — 4 seconds

Yellow LED – Slow / Prepare to Stop — 1.5 seconds

Red LED – Stop — 4 seconds

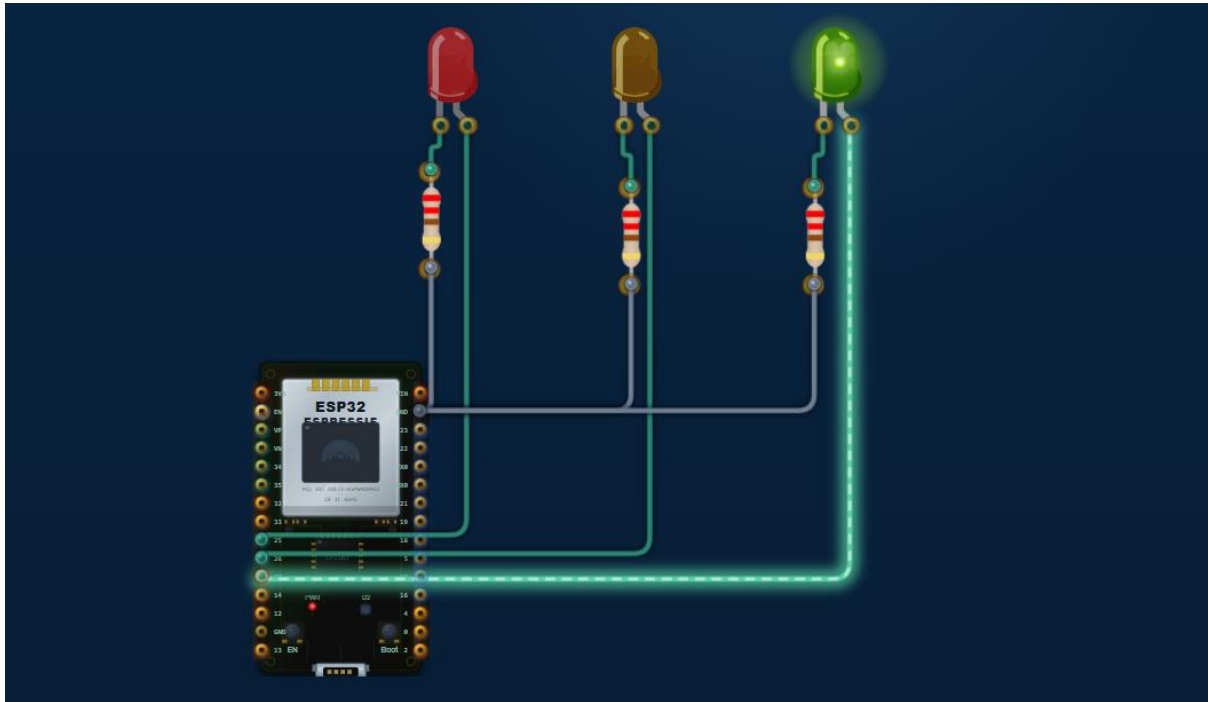
Green LED – Go — 4 seconds

Yellow LED – Slow / Prepare to Stop — 1.5 seconds

⋮

nth term: The sequence (Red LED → Green LED → Yellow LED) repeats continuously.

Circuit Diagram:



Doorbell Tone

PROBLEM:

Pressing the push button connected to pin D2 triggers a classic two-note doorbell sequence (E5 at 659 Hz, then C5 at 523 Hz) on a buzzer connected to pin D8..

CONNECTION:

Push button GPIO 2 ↔ GND

Buzzer (+) GPIO 18

Buzzer (−) GND

CODE:

```
const int BTN_PIN = 2;

const int BUZZ_PIN = 18;


void setup() {

  pinMode(BTN_PIN, INPUT_PULLUP);


  Serial.begin(115200);

  Serial.println("ESP32 Doorbell Ready");

}


void loop() {

  // Button pressed = LOW
```

```
if (digitalRead(BTN_PIN) == LOW) {
```

```
    Serial.println("Ding Dong Ding!");
```

```
    // Ding - E5 (659 Hz)
```

```
    tone(BUZZ_PIN, 659);
```

```
    delay(400);
```

```
    noTone(BUZZ_PIN);
```

```
    delay(100);
```

```
    // Dong - C5 (523 Hz)
```

```
    tone(BUZZ_PIN, 523);
```

```
    delay(600);
```

```
    noTone(BUZZ_PIN);
```

```
    delay(100);
```

```
    // Ding - G5 (784 Hz)
```

```
    tone(BUZZ_PIN, 784);
```

```
    delay(400);
```

```
    noTone(BUZZ_PIN);
```

```
    // 3-second lockout
```

```
    delay(3000);
```

```
}
```

}

OBSERVED OUTPUT:

When the ESP32 is powered on, the Serial Monitor displays:

ESP32 Doorbell Ready

When the push button is pressed:

Ding Dong Ding!

At the same time, the passive buzzer produces three tones:

E5 (659 Hz) → C5 (523 Hz) → G5 (784 Hz)

Ding Dong Ding

After playing the three tones, the system waits **3 seconds** before accepting another button press.

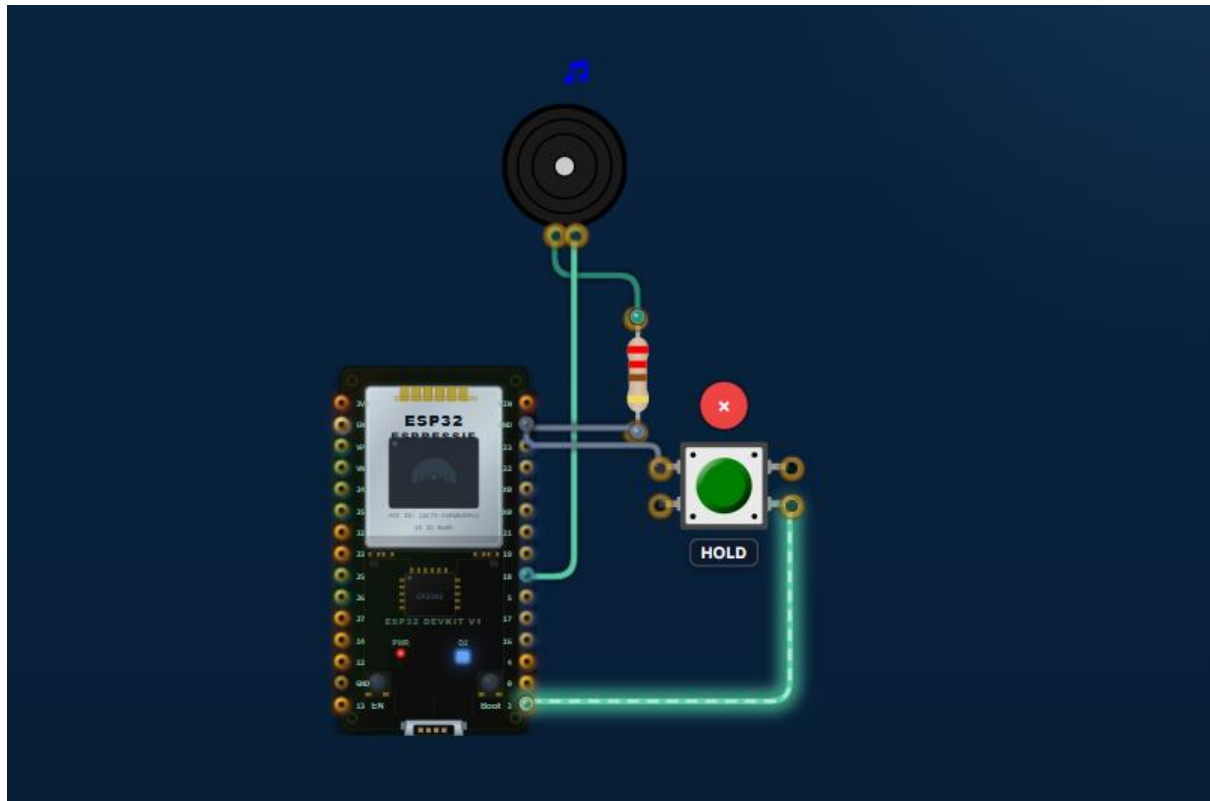
EXPECTED OUTPUT:

Serial Monitor when button is pressed:

Doorbell ready

Ding dong!

Circuit Diagram:



Batch 1B

Analog Meter Serial

PROBLEM:

As you turn the potentiometer knob on the canvas, the Terminal updates every 200 ms with the raw value, the calculated voltage, and the percentage rotation.

CONNECTION:

Potentiometer **ESP32**

Outer pin 1 **3.3V**

Middle pin (wiper) **GPIO 34 (ADC)**

Outer pin 2 **GND**

CODE:

```
const int potPin = 34;

void setup() {
  Serial.begin(115200);
  analogReadResolution(12); // 0-4095
}

void loop() {
  int raw = analogRead(potPin);
```

```
float voltage = (raw / 4095.0) * 3.3;

float percentage = (raw / 4095.0) * 100.0;


Serial.print("Raw: ");

Serial.print(raw);


Serial.print(" Voltage: ");

Serial.print(voltage, 2);

Serial.print(" V");


Serial.print(" Rotation: ");

Serial.print(percentage, 1);

Serial.println("%");


delay(200);

}
```

OBSERVED OUTPUT:

When the potentiometer knob is rotated, the ESP32 reads the analog value through the ADC pin. The Serial Monitor updates every 200 ms and displays the raw ADC value, calculated voltage, and percentage of rotation.

Example output:

Raw: 0 Voltage: 0.00 V Rotation: 0.0%

Raw: 1024 Voltage: 0.82 V Rotation: 25.0%

Raw: 2048 Voltage: 1.65 V Rotation: 50.0%

Raw: 3072 Voltage: 2.47 V Rotation: 75.0%

Raw: 4095 Voltage: 3.30 V Rotation: 100.0%

The raw value, voltage, and rotation percentage increase or decrease according to the direction in which the potentiometer knob is turned.

EXPECTED OUTPUT:

The ESP32 should continuously read the potentiometer position and display the corresponding values in the Serial Monitor every 200 ms.

The Serial Monitor is expected to show:

Raw: 0 Voltage: 0.00 V Rotation: 0.0%

Raw: 1024 Voltage: 0.82 V Rotation: 25.0%

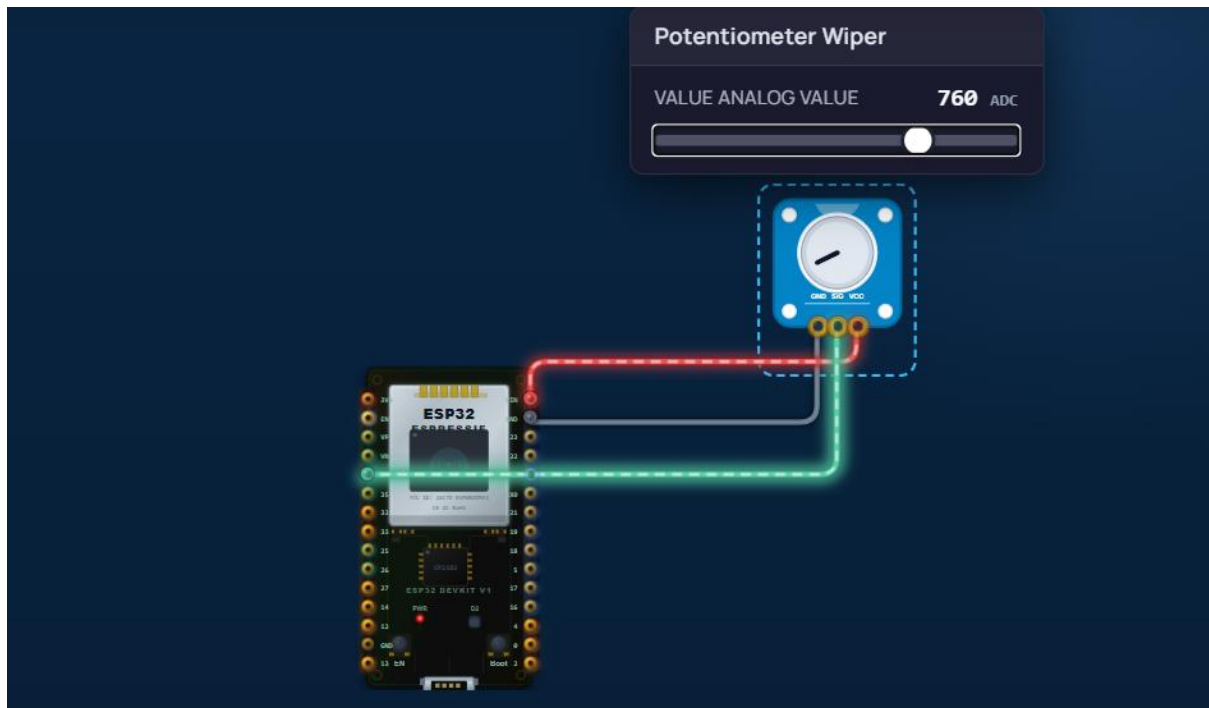
Raw: 2048 Voltage: 1.65 V Rotation: 50.0%

Raw: 3072 Voltage: 2.47 V Rotation: 75.0%

Raw: 4095 Voltage: 3.30 V Rotation: 100.0%

As the potentiometer knob is rotated, the **raw ADC value**, **voltage**, and **percentage rotation** should change accordingly.

Circuit Diagram:



Touch ON/OFF

PROBLEM:

Touching the active sensor pad on the canvas turns the LED on instantly. Releasing your touch turns the LED off.

Why D2 and D13? Pin D2 reads the active-high digital signal from the sensor. Pin D13 drives the visual indicator LED.

CONNECTION:

Touch Sensor **VCC 3.3V**

Touch Sensor **GND GND**

LED **Anode (+)** **GPIO 13** through 220Ω resistor

LED **Cathode (-)** **GND**

CODE:

```
const int sensorPin = 2;
```

```
const int ledPin = 13;
```

```
void setup() {
```

```
    pinMode(sensorPin, INPUT);
```

```
    pinMode(ledPin, OUTPUT);
```

```
}
```

```
void loop() {
```

```
if (digitalRead(sensorPin) == HIGH) {  
    digitalWrite(ledPin, HIGH);  
}  
else {  
    digitalWrite(ledPin, LOW);  
}  
}
```

OBSERVED OUTPUT:

- When the **TTP223 touch sensor is touched**, the ESP32 detects a **HIGH** signal on GPIO 2.
- The **LED connected to GPIO 13 turns ON immediately**.
- The Serial Monitor displays:

Touch Detected - LED ON

- When the sensor is **released**, the LED turns **OFF**.
- The Serial Monitor displays:

Touch Sensor Ready

- The circuit continuously repeats this process for every touch and release.
-

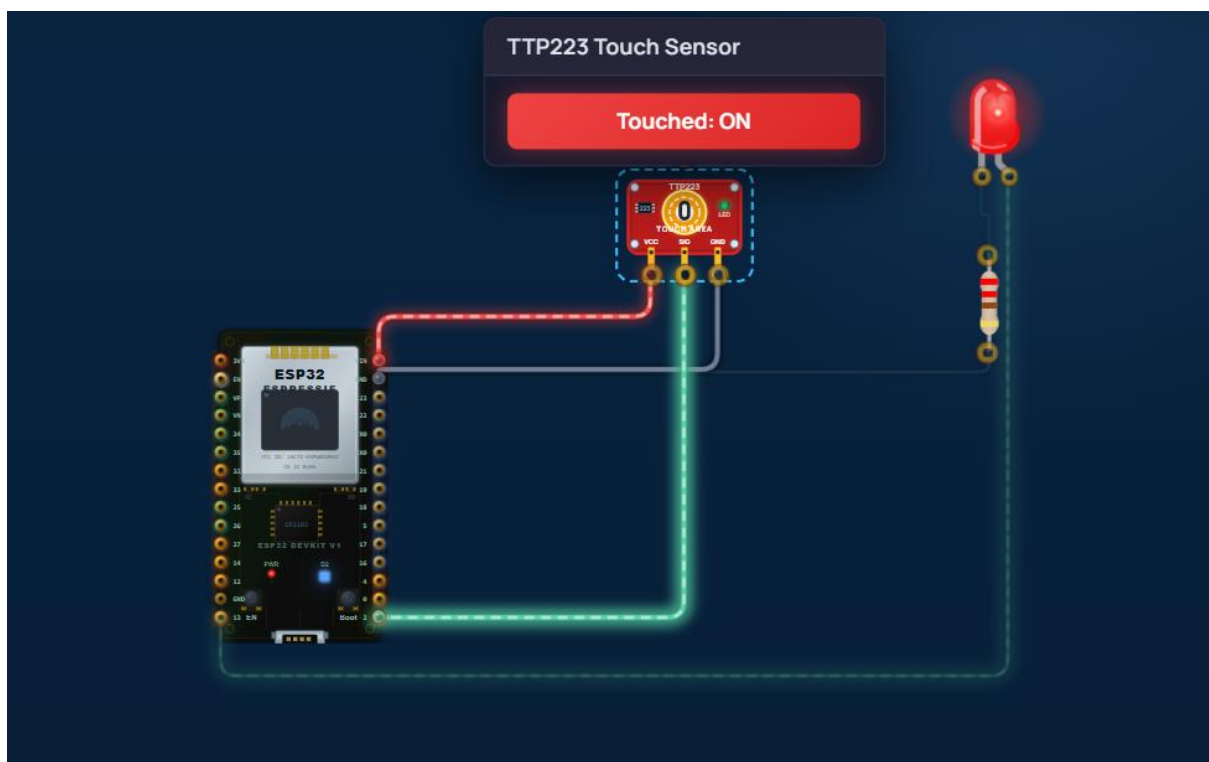
EXPECTED OUTPUT:

- When the **touch sensor is touched**, the LED connected to **GPIO 13** should turn **ON**.
- When the **touch is released**, the LED should turn **OFF**.
- The Serial Monitor should show:
- Touch Sensor Ready

Touch Detected - LED ON

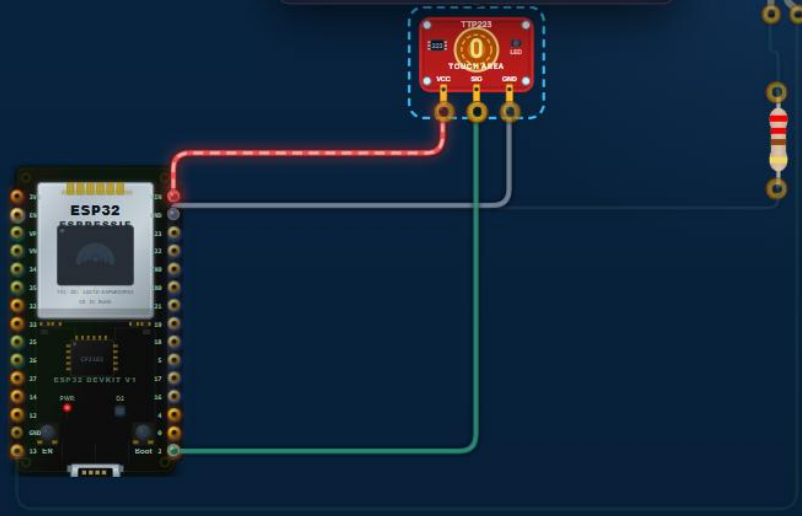
- The ESP32 should continuously monitor the touch sensor and respond immediately to each touch and release.

Circuit Diagram:



TTP223 Touch Sensor

Touched: OFF



Batch 1C

Motion Alert Print

PROBLEM:

When the PIR sensor detects motion, an alert message is printed to the Terminal. When the motion ceases, a quiet message is printed.

Why D2? Pin D2 supports digital input and hardware state checking, making it a standard choice for motion events.

CONNECTION:

VCC 5V / VIN

GND GND

OUT GPIO 2 (D2)

CODE:

```
#define PIR_PIN 2 // GPIO 2 (check your board's D2 mapping)
```

```
void setup() {
```

```
  Serial.begin(115200);
```

```
  pinMode(PIR_PIN, INPUT);
```

```
  Serial.println("PIR sensor ready...");
```

```
}
```

```
void loop() {  
  
  int motionState = digitalRead(PIR_PIN);  
  
  if (motionState == HIGH) {  
    Serial.println("Motion detected!");  
  } else {  
    Serial.println("No motion.");  
  }  
  
  delay(500);  
}
```

OBSERVED OUTPUT:

When the PIR sensor detects motion, the Serial Monitor displays:

PIR sensor ready...

No motion.

No motion.

Motion detected!

Motion detected!

Motion detected!

No motion.

No motion.

- **Motion detected:** Motion detected!

- **Motion stops:** No motion.

Serial Monitor baud rate: **115200**.

EXPECTED OUTPUT:

The Serial Monitor should display:

PIR sensor ready...

No motion.

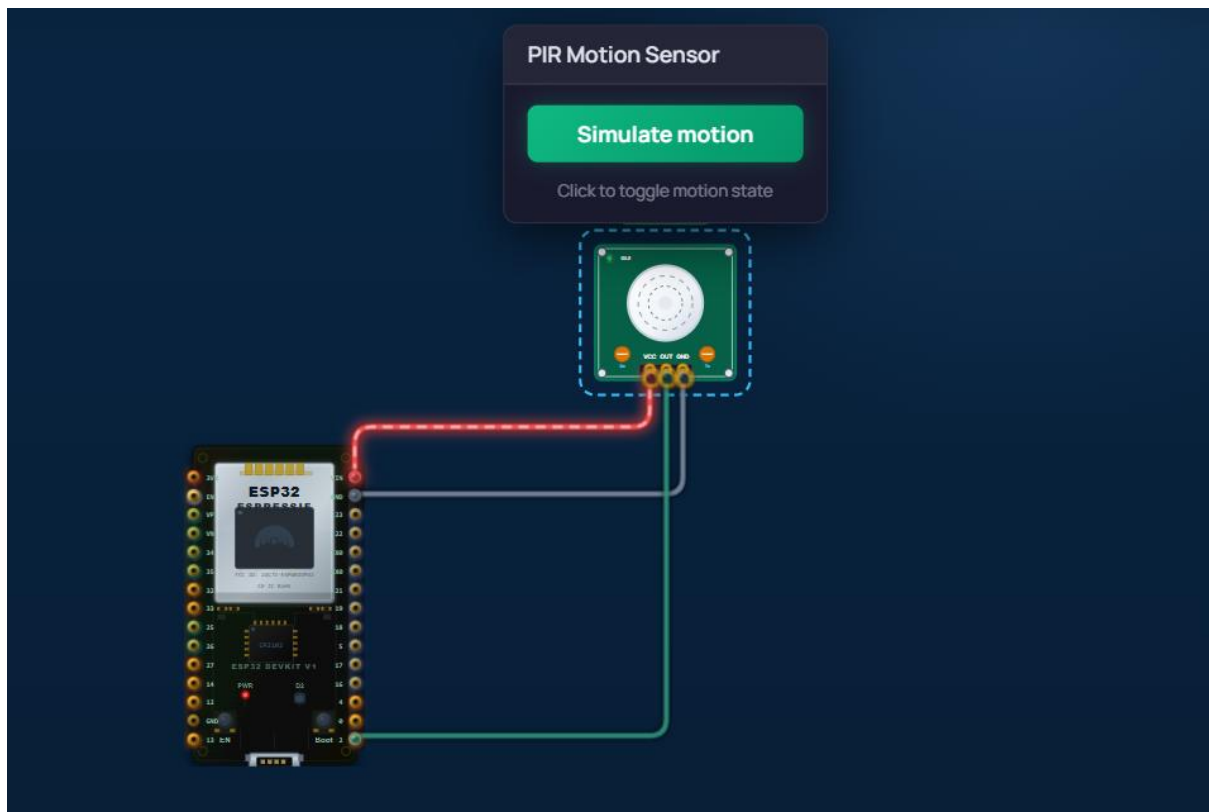
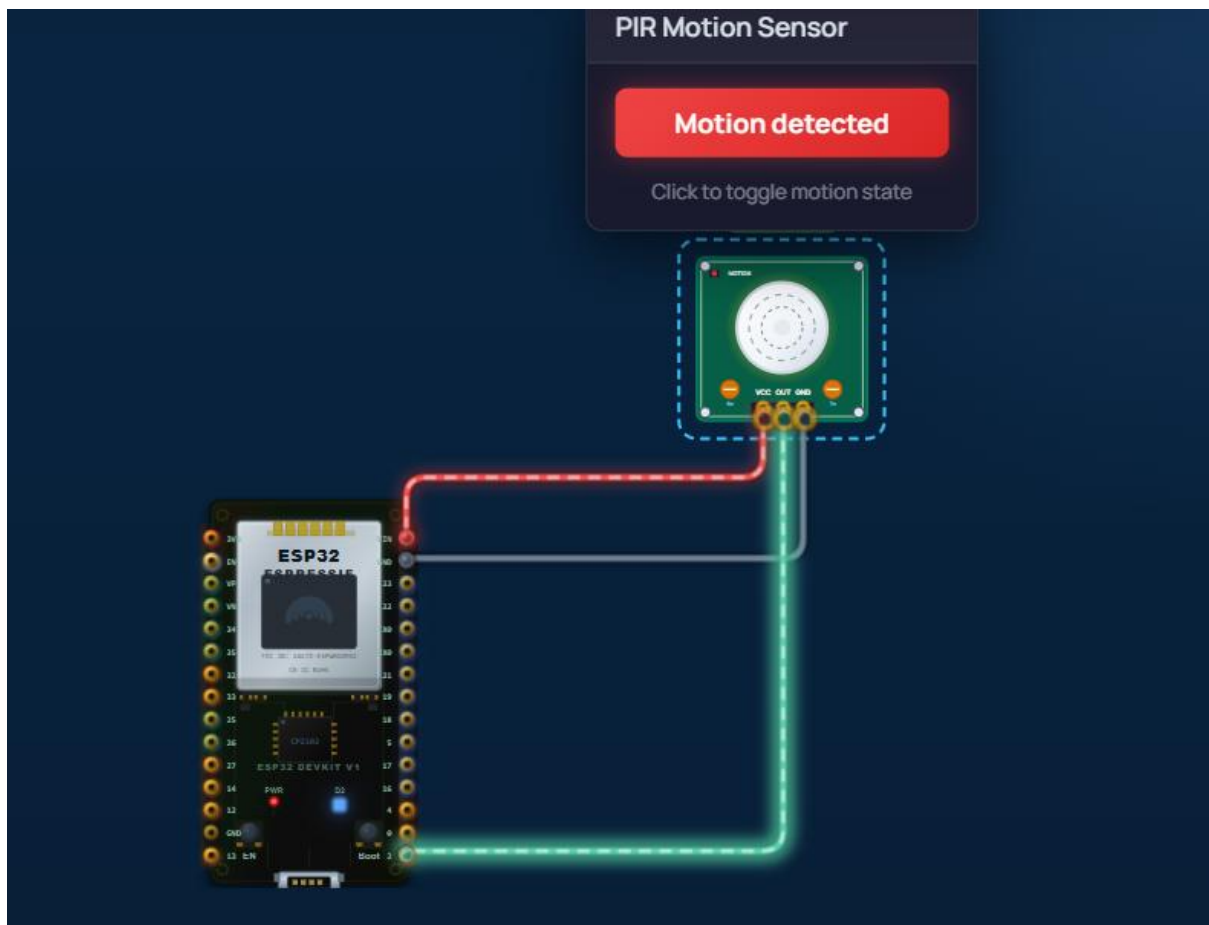
Motion detected!

Motion detected!

No motion.

When the PIR sensor detects movement, “**Motion detected!**” is printed. When movement stops, “**No motion.**” is printed.

Circuit Diagram:



Obstacle Print

PROBLEM:

When an object is placed in front of the IR sensor, it triggers a digital detection signal, and the Terminal prints "[WARNING] Obstacle Detected!". When the object is removed, it prints "[INFO] Path Clear.".

Why D2? Pin D2 reads the digital output of the IR sensor module. D2 is used for standard digital inputs.

CONNECTION:

VCC 3.3V

GND GND

OUT GPIO 2

CODE:

```
#define IR_SENSOR 2

void setup() {
    pinMode(IR_SENSOR, INPUT);
    Serial.begin(115200);
}

void loop() {
    int sensorState = digitalRead(IR_SENSOR);
```

```
if (sensorState == LOW) {  
    Serial.println("[WARNING] Obstacle Detected!");  
} else {  
    Serial.println("[INFO] Path Clear.");  
}  
  
delay(500);  
}
```

OBSERVED OUTPUT:

When the obstacle is placed in front of the IR sensor:

[WARNING] Obstacle Detected!

[WARNING] Obstacle Detected!

[WARNING] Obstacle Detected!

When the obstacle is removed:

[INFO] Path Clear.

[INFO] Path Clear.

[INFO] Path Clear.

The output is displayed in the **Serial Monitor at 115200 baud**.

EXPECTED OUTPUT:

When an obstacle is placed in front of the IR sensor:

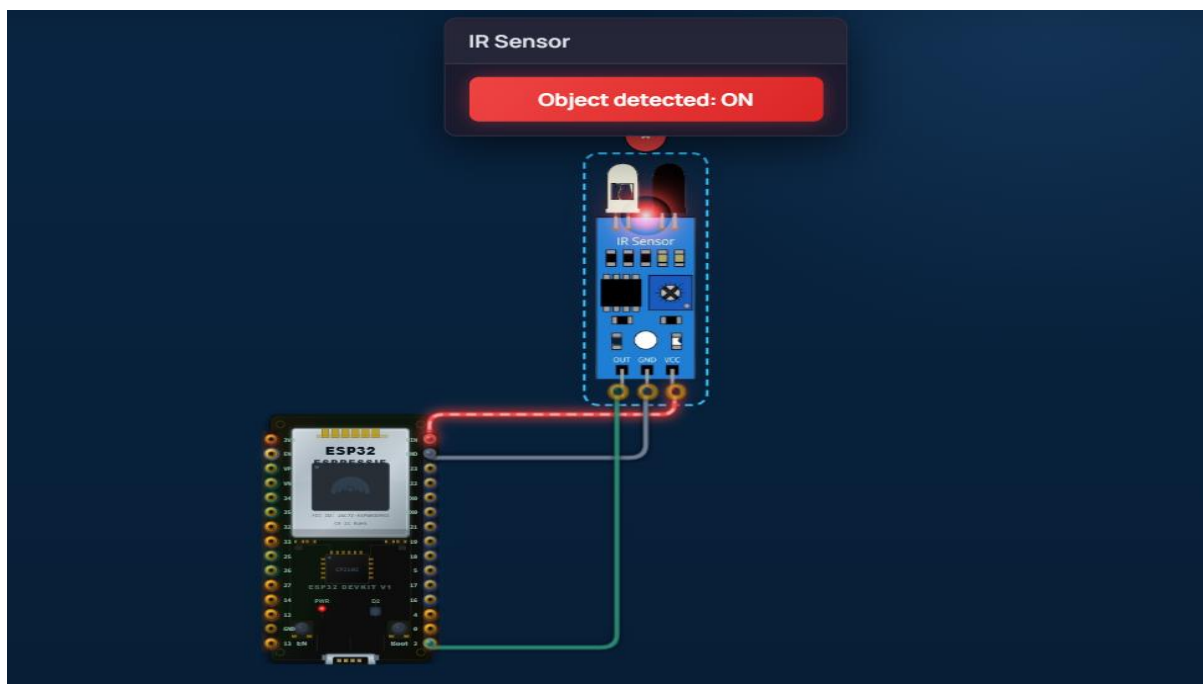
[WARNING] Obstacle Detected!

When the obstacle is removed:

[INFO] Path Clear.

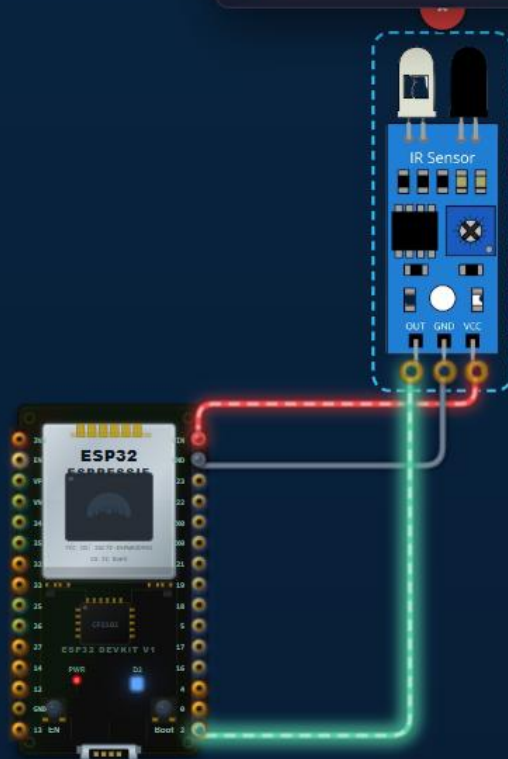
The messages should appear in the **ESP32 Serial Monitor at 115200 baud.**

Circuit Diagram:



IR Sensor

Object detected: OFF



Gas Level Print

PROBLEM:

As you adjust the gas concentration slider on the MQ-2 sensor, the Terminal prints the raw ADC value and a human-readable air quality category (Good, Moderate, Poor, or Danger) every 500 ms.

Why A0? The MQ-2 sensor output voltage changes continuously based on gas concentration, requiring connection to analog pin A0.

CONNECTION:

VCC 5V/VIN*

GND GND

A0 GPIO 34

CODE:

```
const int gasSensorPin = 34;
```

```
void setup() {  
    Serial.begin(115200);  
    delay(1000);  
}
```

```
void loop() {  
    int gasValue = analogRead(gasSensorPin);
```

```
Serial.print("Raw ADC Value: ");

Serial.print(gasValue);

Serial.print(" | Air Quality: ");


if (gasValue < 1000) {

    Serial.println("Good");

}

else if (gasValue < 2000) {

    Serial.println("Moderate");

}

else if (gasValue < 3000) {

    Serial.println("Poor");

}

else {

    Serial.println("Danger");

}


delay(500);

}
```

OBSERVED OUTPUT:

When the MQ-2 gas concentration slider is adjusted, the ESP32 Serial Monitor displays the raw ADC value and corresponding air-quality category every 500 ms.

Example:

Raw ADC Value: 624 | Air Quality: Good

Raw ADC Value: 1187 | Air Quality: Moderate

Raw ADC Value: 2145 | Air Quality: Poor

Raw ADC Value: 3268 | Air Quality: Danger

Raw ADC Value: 2750 | Air Quality: Poor

The **raw ADC value changes continuously** as the gas concentration changes, and the displayed air-quality category changes according to the programmed threshold ranges.

EXPECTED OUTPUT:

The Serial Monitor should display the **raw ADC value from the MQ-2 sensor** along with the corresponding **air quality category** every 500 ms.

Raw ADC Value: 500 | Air Quality: Good

Raw ADC Value: 1200 | Air Quality: Moderate

Raw ADC Value: 2100 | Air Quality: Poor

Raw ADC Value: 3200 | Air Quality: Danger

As the gas concentration increases, the **ADC value should increase**, and the air-quality status should change accordingly:

Good → Moderate → Poor → Danger

Circuit Diagram:

